

RL for Reasoning

Training on answers a program can check · video 2:27

This lesson answers one question: how can a model improve with no human judgements at all? When a program can check the answer (math, code, puzzles), the check itself is the reward. **GRPO** samples a group of answers per question and pushes up the ones that beat their group's average. On Qwen2.5-0.5B, 30 steps take the reward from 0 to 94 out of 100, but the arithmetic was already there: what RL taught was the answer format and when to stop.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Verifiable rewards	▶ 0:08-0:39	2	☐
2. GRPO: the group is the baseline	▶ 0:39-1:13	3	☐
3. What the training actually changed	▶ 1:13-2:07	3	☐

Before you start: Deep Dive episodes 37 (reward models), 38 (RLHF with PPO) and 39 (DPO). Code: `code/d40_rl_reasoning` (PyTorch + transformers; about 55 minutes and 10 GB of memory on a CPU).

CONCEPT 1 OF 3

Verifiable rewards

▶ Video 0:08–0:39

RLHF and DPO learn from human preferences. For tasks whose answer a program can check (an arithmetic result, code that passes tests, a solved puzzle), there is a cheaper signal: the check itself. Training with such a **verifiable reward** needs no reward model and no labels, and the model cannot fool a learned judge, because there is none. This is how reasoning models such as DeepSeek-R1 were trained.

preferences: people judge

math, code, puzzles: a program checks

reinforcement learning on the check alone → reasoning models (DeepSeek-R1)

▶ 0:20 Preferences need people; checkable answers need only a program.

Qwen2.5-0.5B-Instruct

What is $37 \times 2 + 48$? Think step by step, then end with 'Answer: <number>'.

reward = 1 if "Answer: 122"
reward = 0 otherwise

no reward model · no human labels

▶ 0:38 The task and the reward: 1 for the right "Answer: N", else 0.

Our version: Qwen2.5-0.5B-Instruct answers questions like “What is $37 \times 2 + 48$?”, asked to think step by step and end with “Answer: <number>”. The reward is 1 if that final number is right, 0 otherwise: strict about both correctness and format.

KEY IDEA

When answers can be checked automatically, the checker is the reward: no reward model, no human labels.

Check 1: Verifiable rewards

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Which task is the best fit for a verifiable reward?

- A. Writing a function that must pass unit tests
- B. Writing a friendly email
- C. Summarising a news article

1.2 SHORT ANSWER

Our reward also requires the exact text “Answer: N”. What does that strictness reward, besides correct arithmetic?

.....

CONCEPT 2 OF 3

GRPO: the group is the baseline

▶ Video 0:39-1:13

For each question, sample a group of $G = 8$ answers and score them. Each answer's **advantage** is its reward minus the group's mean, divided by the group's standard deviation. The loss raises the log-probability of every token in answers with positive advantage and lowers it for negative ones, plus a KL penalty ($\beta = 0.04$) that keeps the model near where it started.

GRPO: group relative policy optimization

8 answers to one question, rewards

1	0	1	1	0	0	1	0
+0.9	-0.9	+0.9	+0.9	-0.9	-0.9	+0.9	-0.9

$$\text{advantage} = (\text{reward} - \text{group mean}) / \text{group spread}$$

better than its siblings: more likely
worse: less likely

no value head: the group is the baseline

▶ 0:58 Eight answers to one question: advantage = (reward – group mean) / group spread.

1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0
1	0	0	1	1	0	1	1

all right: advantages 0
all wrong: advantages 0
mixed: a signal

+ a small KL penalty ($\beta = 0.04$) keeps it near the start

▶ 1:12 All right or all wrong: every advantage is 0, no signal.

PPO (episode 38) needs a value head to estimate how good a situation is; GRPO uses the other answers to the same question instead. The price: if all eight answers get the same reward, every advantage is 0 and that question teaches nothing. Learning needs questions the model sometimes gets right and sometimes wrong.

```
1 mean = r.mean(1, keepdim=True) # r: one row per group
2 std = r.std(1, keepdim=True)
3 adv = (r - mean) / (std + 1e-4)
4 loss = -adv * logp + beta * kl
```

▶ 2:18 The core of GRPO in code.

KEY IDEA

Each answer is judged against its siblings: better than the group average → more likely; worse → less likely.

Check 2: GRPO: the group is the baseline

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

A group's rewards are 1, 0, 1, 1, 0, 0, 1, 0. What is the advantage of each right answer? (Use the sample standard deviation.)

2.2 CALCULATION

Seven answers right, one wrong. What are the advantages?

2.3 TRUE OR FALSE

"A question the model always gets right still improves it under GRPO."

True ☐ False ☐

CONCEPT 3 OF 3

What the training actually changed

▶ Video 1:13-2:07

Before training, on 100 held-out questions (greedy): reward 0. Yet the last number in the text is right 95 times: the model computes correctly, then ends with `\boxed{...}` instead of “Answer: N”, and 16 answers run past the 160-token limit.

before training · 100 held-out questions, greedy

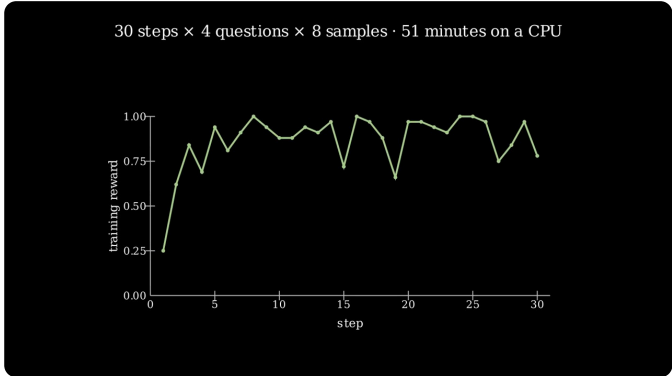
reward: 0 / 100

last number in the text right: 95 / 100

...2. Addition: 74 + 48 = 122
Therefore, the answer is: \boxed{

16 / 100 run out of room (160 tokens)

▶ 1:28 Before: reward 0/100, but the last number is right 95/100.



▶ 1:42 Training reward per step: 0.25 → 0.94 by step 5, then 0.66–1.00.

	BEFORE	AFTER GRPO
reward (“Answer: N” right)	0	94
finished within 160 tokens	84	100
mean length (tokens)	148	105
last number right	95	94

	before	after GRPO
reward (“Answer: N” right)	0	94
finished within 160 tokens	84	100
mean length (tokens)	148	105
last number right	95	94

...2. Next, add 48: 74 + 48 = 122
Therefore, the answer is:
Answer: 122

it learned the form the reward checks, not arithmetic
at scale: longer chains of thought that check their own work

▶ 2:06 After: 94/100; every answer finishes, 29% shorter; arithmetic unchanged.

So RL did not teach arithmetic here: it taught the model to use what it already knew in the form the reward checks, and to stop. That is typical: RL mostly *amplifies* behaviour the model already produces sometimes (sampled answers already scored 0.25 at step 1). In large models, the same pressure has grown longer chains of thought that check their own work.

KEY IDEA

RL with a verifiable reward strengthens what the model can already do some of the time; check which behaviour actually changed.

Check 3: What the training actually changed

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Mean length fell from 148 to 105 tokens. By what percentage?

.....

3.2 SHORT ANSWER

Why could GRPO make progress from the very first step, even though greedy decoding scored 0?

.....

3.3 HANDS-ON CODE

Try it yourself. Compute the GRPO advantages for 4 groups of 8 rewards.

```
import torch
r = torch.tensor([[1, 0, 1, 1, 0, 0, 1, 0],
                  [1, 1, 1, 1, 1, 1, 1, 0],
                  [0, 0, 0, 0, 0, 0, 0, 0],
                  [1, 1, 1, 1, 1, 1, 1, 1]], dtype=torch.float)
adv = (r - r.mean(1, keepdim=True)) / (r.std(1, keepdim=True) + 1e-4)
print(adv.round(decimals=2))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Verifiable rewards · rewatch ▶ 0:08-0:39

1.1 Writing a function that must pass unit tests. Running the tests checks the answer automatically; the other two need a judge.

1.2 Following the requested output format. Before training the model was usually right but used a different format, so it scored 0.

2. GRPO: the group is the baseline · rewatch ▶ 0:39-1:13

2.1 About +0.94. Mean 0.5; standard deviation 0.53; $(1 - 0.5) / 0.53 \approx 0.94$.

2.2 About +0.35 for each right answer, -2.47 for the wrong one. Mean 0.875, standard deviation 0.354: a rare failure gets a large push down.

2.3 False. All rewards equal → all advantages 0; only the KL term acts.

3. What the training actually changed · rewatch ▶ 1:13-2:07

3.1 About 29%. $1 - 105 / 148 \approx 0.29$.

3.2 Sampled answers sometimes used the right format (training reward 0.25 at step 1), so groups had mixed results and non-zero advantages. GRPO can only amplify behaviour that appears in the samples.

3.3 Rows: ±0.94; +0.35 and -2.47; all 0; all 0. The last two groups give no learning signal.